

Aim

To write a Java program that demonstrates the NavigableMap interface methods lowerKey(), floorKey(), ceilingKey(), higherKey(), firstEntry(), lastEntry(), pollFirstEntry(), pollLastEntry() and descendingMap() using a TreeMap.

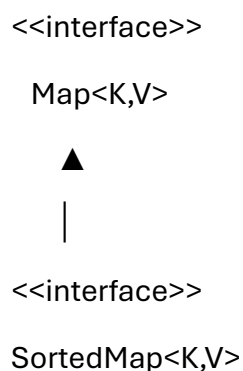
Algorithm

1. Start.
2. Import the java.util package.
3. Create a NavigableMap<Integer, String> object using TreeMap, mapping student ID to student name.
4. Add the pairs (10, 20, 30, 40, 50) using put() and display the map.
5. Find keys relative to a given key:
 - lowerKey(k) returns the greatest key strictly less than k.
 - floorKey(k) returns the greatest key less than or equal to k.
 - ceilingKey(k) returns the least key greater than or equal to k.
 - higherKey(k) returns the least key strictly greater than k.

Each returns null if no such key exists.

6. Display the entries with the least and greatest keys using firstEntry() and lastEntry(). The map is not changed.
7. Display the reverse-order view of the map using descendingMap().
8. Remove and display the first and last entries using pollFirstEntry() and pollLastEntry().
9. Display the map after the removals.
10. Stop.

UML Diagram




```

import java.util.*;

public class NavigableMapInterface {

    public static void main(String[] args) {

        NavigableMap<Integer, String> nm = new TreeMap<>();

        nm.put(30, "Chitra");
        nm.put(10, "Arun");
        nm.put(50, "Eswar");
        nm.put(20, "Bala");
        nm.put(40, "Dinesh");

        System.out.println("NavigableMap      : " + nm);

        // lowerKey() - strictly less than
        System.out.println("lowerKey(30)      : " + nm.lowerKey(30));
        System.out.println("lowerKey(10)      : " + nm.lowerKey(10));

        // floorKey() - less than or equal
        System.out.println("floorKey(30)     : " + nm.floorKey(30));
        System.out.println("floorKey(35)     : " + nm.floorKey(35));
        System.out.println("floorKey(5)      : " + nm.floorKey(5));

        // ceilingKey() - greater than or equal
        System.out.println("ceilingKey(30)   : " + nm.ceilingKey(30));
        System.out.println("ceilingKey(35)   : " + nm.ceilingKey(35));
        System.out.println("ceilingKey(55)   : " + nm.ceilingKey(55));

        // higherKey() - strictly greater than
    }
}

```

```
System.out.println("higherKey(30)    : " + nm.higherKey(30));
System.out.println("higherKey(50)    : " + nm.higherKey(50));

// firstEntry() and lastEntry()
System.out.println("firstEntry()     : " + nm.firstEntry());
System.out.println("lastEntry()      : " + nm.lastEntry());

// descendingMap()
System.out.println("descendingMap()   : " + nm.descendingMap());

// pollFirstEntry() and pollLastEntry()
System.out.println("pollFirstEntry()  : " + nm.pollFirstEntry());
System.out.println("pollLastEntry()   : " + nm.pollLastEntry());
System.out.println("Map after polling : " + nm);
}
}
```

Output:

```

PS C:\Users\Sahithi_pyla\Desktop\Sahithi_java> & 'C:\Program Files\Java\jdk-20
es' '-cp' 'C:\Users\Sahithi_pyla\AppData\Roaming\Code\User\workspaceStorage\234
bservations.NavigableMapInterface'
NavigableMap          : {10=Arun, 20=Bala, 30=Chitra, 40=Dinesh, 50=Eswar}
lowerKey(30)          : 20
lowerKey(10)           : null
floorKey(30)           : 30
floorKey(35)           : 30
floorKey(5)            : null
ceilingKey(30)         : 30
ceilingKey(35)         : 40
ceilingKey(55)         : null
higherKey(30)          : 40
higherKey(50)          : null
firstEntry()           : 10=Arun
lastEntry()            : 50=Eswar
descendingMap()        : {50=Eswar, 40=Dinesh, 30=Chitra, 20=Bala, 10=Arun}
pollFirstEntry()       : 10=Arun
pollLastEntry()        : 50=Eswar
Map after polling      : {20=Bala, 30=Chitra, 40=Dinesh}
PS C:\Users\Sahithi_pyla\Desktop\Sahithi_java>

```

Result:

Thus, the Java program demonstrating the NavigableMap interface methods was executed successfully and the output was verified.

- lowerKey() and higherKey() are strict comparisons, while floorKey() and ceilingKey() also accept an exact match.
- All four navigation methods return null when no suitable key exists.
- firstEntry() and lastEntry() only read the map, while pollFirstEntry() and pollLastEntry() remove the entry and return it.
- descendingMap() is a live reverse-order view of the original map, not a copy.

The entries returned by firstEntry(), lastEntry() and the poll methods are read-only snapshots. Calling setValue() on them throws UnsupportedOperationException